

Pose Estimation

Pose Estimation

1. Model Introduction
2. Pose Estimation: Image
Effect Preview
3. Pose Estimation: Video
Effect Preview
5. Pose Estimation: Real-time Detection (CSI Camera)
Effect Preview
4. Pose Estimation: Real-time Detection (USB Camera)
Effect Preview

Use Python to demonstrate **Ultralytics**: Pose Estimation effects on images, videos, and real-time detection.

1. Model Introduction

Pose estimation is a task involving identifying the location of specific points in images (usually called keypoints). Keypoints can represent various parts of objects, such as joints, landmarks, or other significant features. The location of keypoints is usually represented by a set of 2D `[x, y]` or 3D `[x, y, visible]` coordinates

The output of a pose estimation model is a set of points representing object keypoints in the image, usually including confidence scores for each point. Pose estimation is a good choice when you need to identify specific parts of objects in a scene and their positional relationships to each other.

In the default YOLO26 pose model, there are 17 keypoints, each representing different parts of the human body. Here is the mapping of each index to the corresponding body joint:

1. Nose
2. Left eye
3. Right eye
4. Left ear
5. Right ear
6. Left shoulder
7. Right shoulder
8. Left elbow
9. Right elbow
10. Left wrist
11. Right wrist
12. Left hip
13. Right hip
14. Left knee
15. Right knee
16. Left ankle
17. Right ankle

2. Pose Estimation: Image

Use yolo26n-pose_ncnn_model to predict images in the ultralytics26 folder (not images that come with ultralytics).

Enter the code folder:

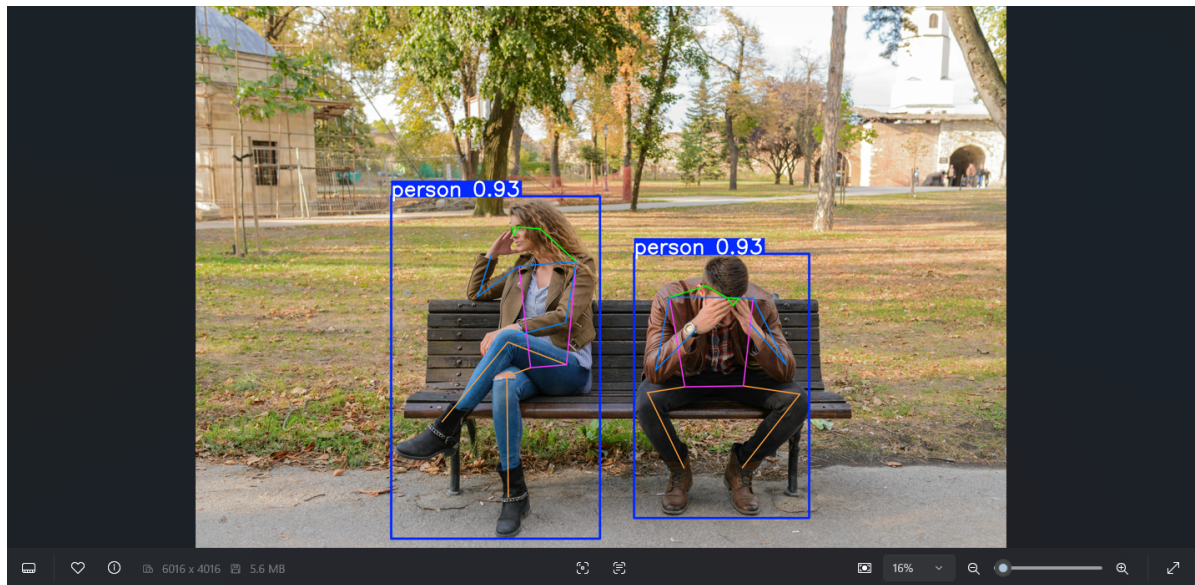
```
cd /ultralytics/yahboom_demo
```

Run the code:

```
python3 03.pose_image.py
```

Effect Preview

yolo recognition output image location: /root/yolo26_data/output/



Sample code:

```
from ultralytics import YOLO

# Load a model
model = YOLO("/root/yolo26_data/models/yolo26n-pose.pt")

# Run batched inference on a list of images
results = model("/root/yolo26_data/people.jpg") # return a list of Results
objects

# Process results list
for result in results:
    # boxes = result.boxes # Boxes object for bounding box outputs
    # masks = result.masks # Masks object for segmentation masks outputs
    keypoints = result.keypoints # Keypoints object for pose outputs
    # probs = result.probs # Probs object for classification outputs
    # obb = result.obb # Oriented boxes object for OBB outputs
    result.show() # display to screen
    result.save(filename="/root/yolo26_data/output/people_output.jpg") # save
    to disk
```

3. Pose Estimation: Video

Use yolo26n-pose_ncnn_model to predict videos in the ultralytics26 folder (not videos that come with ultralytics).

Enter the code folder:

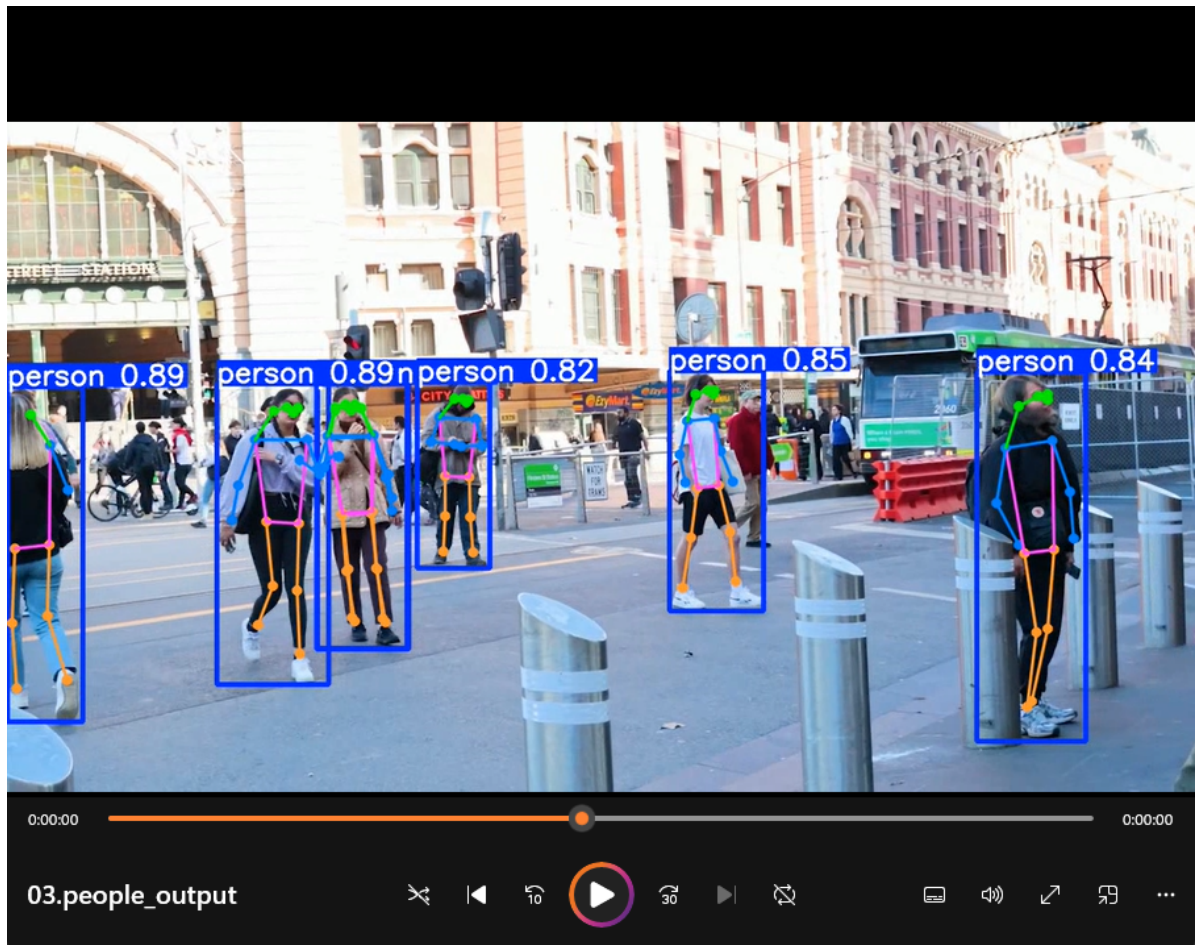
```
cd /ultralytics/yahboom_demo
```

Run the code:

```
python3 03.pose_video.py
```

Effect Preview

yolo recognition output video location: /root/yolo26_data/output/



Sample code:

```
import cv2
from ultralytics import YOLO

# Load the YOLO model
model = YOLO("/root/yolo26_data/models/yolo26n-pose.pt")

# Open the video file
video_path = "/root/yolo26_data/people_s.mp4"
cap = cv2.VideoCapture(video_path)

# Get the video frame size and frame rate
frame_width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
frame_height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
```

```

fps = int(cap.get(cv2.CAP_PROP_FPS))

# Define the codec and create a Videowriter object to output the processed video
output_path = "/root/yolo26_data/output/03.people_output.mp4"
fourcc = cv2.VideoWriter_fourcc(*'mp4v') # You can use 'XVID' or 'mp4v'
depending on your platform
out = cv2.VideoWriter(output_path, fourcc, fps, (frame_width, frame_height))

# Loop through the video frames
while cap.isOpened():
    # Read a frame from the video
    success, frame = cap.read()

    if success:
        # Run YOLO inference on the frame
        results = model(frame)

        # Visualize the results on the frame
        annotated_frame = results[0].plot()

        # Write the annotated frame to the output video file
        out.write(annotated_frame)

        # Display the annotated frame
        cv2.imshow("YOLO Inference", cv2.resize(annotated_frame, (640, 480)))

        # Break the loop if 'q' is pressed
        if cv2.waitKey(1) & 0xFF == ord("q"):
            break
    else:
        # Break the loop if the end of the video is reached
        break

# Release the video capture and writer objects, and close the display window
cap.release()
out.release()
cv2.destroyAllWindows()

```

5. Pose Estimation: Real-time Detection (CSI Camera)

Use yolo26n-pose for real-time pose estimation through CSI camera.

Note: Please ensure CSI camera is properly connected and use docker startup script that supports CSI camera to enter container

Enter the code folder:

```
cd /ultralitics/yahboom_demo
```

Run the code:

```
python3 03.pose_camera_csi.py
```

Press keyboard **q** key to exit real-time detection

Effect Preview

yolo recognition output video location: /root/yolo26_data/output/

Sample code:

```
import cv2
from ultralytics import YOLO

def gstreamer_pipeline(sensor_id=0, width=640, height=480, fps=30):
    return (
        f"nvarguscamerasrc sensor-id={sensor_id} ! "
        f"video/x-raw(memory:NVMM),width={width},height={height},framerate=
{fps}/1 ! "
        "nvvidconv ! "
        "video/x-raw,format=BGRx ! "
        "videoconvert ! "
        "video/x-raw,format=BGR ! appsink"
    )

# Load the YOLO model
model = YOLO("/root/yolo26_data/models/yolo26n-pose.pt")

# Open the camera
frame_width = 640
frame_height = 480
fps = 30
cap = cv2.VideoCapture(
    gstreamer_pipeline(width=frame_width, height=frame_height, fps=fps),
    cv2.CAP_GSTREAMER,
)

if not cap.isOpened():
    raise RuntimeError("Failed to open CSI camera")

# Define the codec and create a VideoWriter object to output the processed video
output_path = "/root/yolo26_data/output/03.pose_camera_csi.mp4"
fourcc = cv2.VideoWriter_fourcc(*"mp4v")
out = cv2.VideoWriter(output_path, fourcc, fps, (frame_width, frame_height))

# Loop through the video frames
while cap.isOpened():
    # Read a frame from the video
    success, frame = cap.read()

    if success:
        # Run YOLO inference on the frame
        results = model(frame)

        # Visualize the results on the frame
        annotated_frame = results[0].plot()

        # Write the annotated frame to the output video file
        out.write(annotated_frame)
```

```

# Display the annotated frame
cv2.imshow("YOLO Inference", cv2.resize(annotated_frame, (640, 480)))

# Break the loop if 'q' is pressed
if cv2.waitKey(1) & 0xFF == ord("q"):
    break
else:
    # Break the loop if the camera frame cannot be read
    break

# Release the video capture and writer objects, and close the display window
cap.release()
out.release()
cv2.destroyAllWindows()

```

4. Pose Estimation: Real-time Detection (USB Camera)

Use yolo26n-pose for real-time pose estimation through USB camera.

Note: Please ensure USB camera is properly connected to JetsonNano

Enter the code folder:

```
cd /ultralytics/yahboom_demo
```

Run the code:

```
python3 03.pose_camera_usb.py
```

Press keyboard **q** key to exit real-time detection

Effect Preview

yolo recognition output video location: /root/yolo26_data/output/

Sample code:

```

import cv2
from ultralytics import YOLO

# Load the YOLO model
model = YOLO("/root/yolo26_data/models/yolo26n-pose.pt")

# Open the camera
cap = cv2.VideoCapture(0)
cap.set(6, cv2.VideoWriter_fourcc('M', 'J', 'P', 'G'))
cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)

# Get the video frame size and frame rate
frame_width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
frame_height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
fps = int(cap.get(cv2.CAP_PROP_FPS))

```

```

# Define the codec and create a Videowriter object to output the processed video
output_path = "/root/yolo26_data/output/03.pose_camera_usb.mp4"
fourcc = cv2.VideoWriter_fourcc(*'mp4v') # You can use 'XVID' or 'mp4v'
depending on your platform
out = cv2.VideoWriter(output_path, fourcc, fps, (frame_width, frame_height))

# Loop through the video frames
while cap.isOpened():
    # Read a frame from the video
    success, frame = cap.read()

    if success:
        # Run YOLO inference on the frame
        results = model(frame)

        # Visualize the results on the frame
        annotated_frame = results[0].plot()

        # Write the annotated frame to the output video file
        out.write(annotated_frame)

        # Display the annotated frame
        cv2.imshow("YOLO Inference", cv2.resize(annotated_frame, (640, 480)))

        # Break the loop if 'q' is pressed
        if cv2.waitKey(1) & 0xFF == ord("q"):
            break
    else:
        # Break the loop if the end of the video is reached
        break

# Release the video capture and writer objects, and close the display window
cap.release()
out.release()
cv2.destroyAllWindows()

```